

Open in app ↗



Medium



Search



Write



#30DAYSOFNLP

NLP-Day 21: Understanding Transformer Models And Architecture

Dissecting the transformer model and architecture to gain a deeper understanding



Marvin Lanhenke

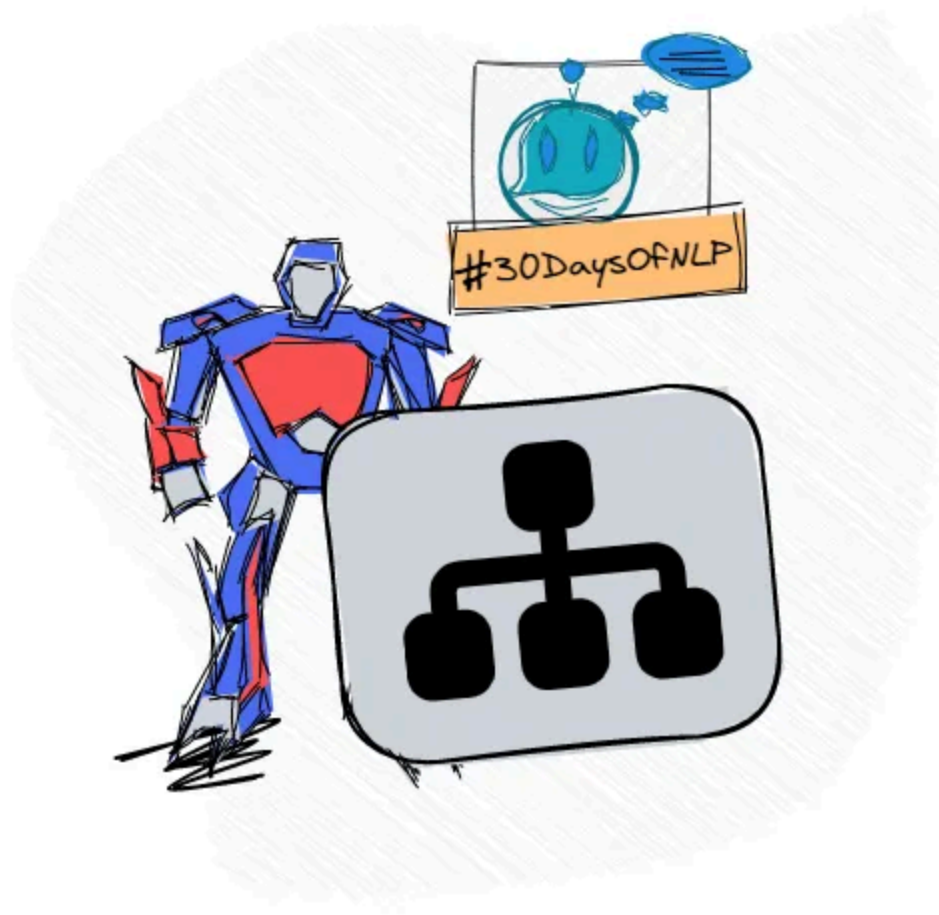
Follow

5 min read · Apr 27, 2022



4





Transformer-based architecture #30DaysOfNLP [Image by Author]

In the last episode, we strengthened our foundation by covering the attention mechanism in general as well as in the context of transformer-based architectures.

By encountering the concept of self-attention, we made the first important step towards a deeper understanding of transformer-based neural networks.

Now, it's time to continue our journey.

In the following sections, we're going to dissect the transformer model and architecture, hopefully equipping us with the necessary tools and knowledge to be able to implement a transformer-based network ourselves.

So take a seat, don't go anywhere, and make sure to follow #30DaysOfNLP:
You Better Pay Attention To Transformers (Part 3)

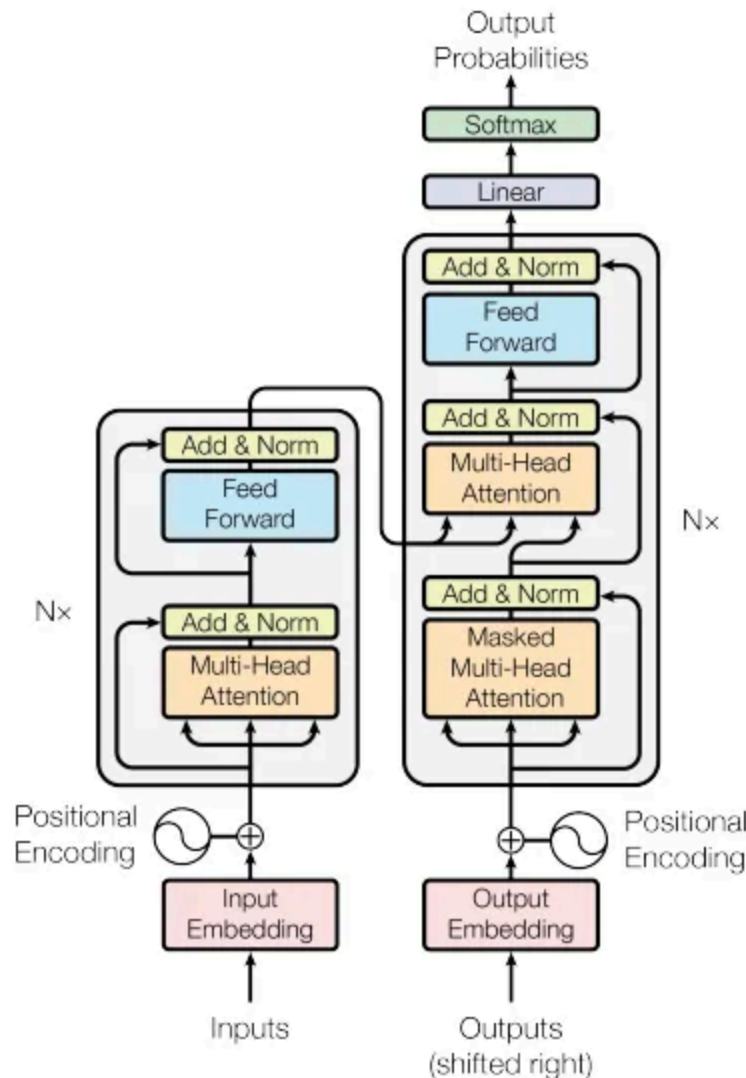
Note: The following parts are based on the explanations provided in the paper Attention Is All You Need, which is basically a must-read for everyone interested in Transformers.

The architecture

So far we already introduced the concepts of attention and in particular self-attention as implemented by the Transformer attention mechanism.

Now, it's time to slightly shift our focus on the Transformer architecture itself.

The Transformer architecture is based on an encoder-decoder structure, however, without relying on recurrence and convolutions.

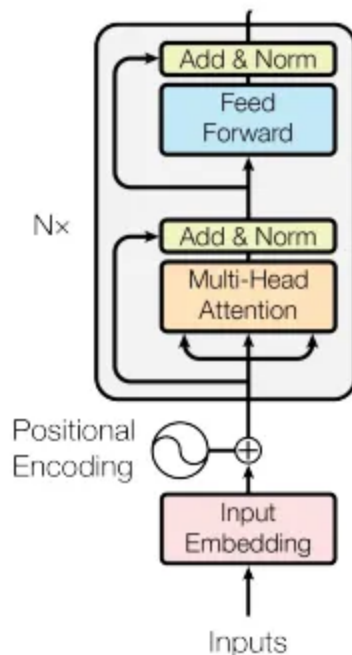


The encoder-decoder structure [Image taken from [Attention Is All You Need](#), Vaswani et al. (2017)]

Boiled down to its very essence, the architecture consists of an encoder that maps an input sequence to a sequence of continuous representations, which is then fed into a decoder. The decoder receives this input together with the decoder's previous output and generates the final output sequence.

It's time to encode

The encoder consists of a stack of 6 identical layers, where each layer is composed of two sublayers.



The encoder [Image taken from [Attention Is All You Need](#), Vaswani et al. (2017)]

The first sublayer implements a multi-head self-attention mechanism that receives n -different linear projections of the query, keys, and values. Each projected version produces n -different outputs in parallel in order to generate the final result of this sublayer.

The second sublayer is defined by a fully connected feed-forward network. The network contains two linear transformation layers with a ReLU activation in between.

A residual connection is applied around each sublayer as well as a normalization layer.

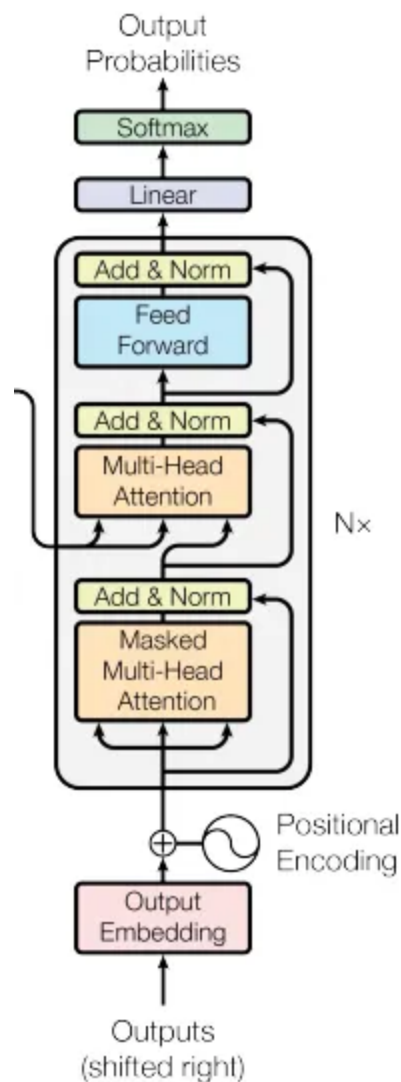
Due to the Transformer's inherent structure, we cannot capture information about the relative position of the words in the sequence. Thus, we compute a positional encoding by using sine and cosine functions of different

frequencies. We inject this positional encoding by simply summing it to the input embeddings.

The decoder

The decoder is quite similar to the encoder.

It also consists of a stack of 6 identical layers. However, this time each layer is composed of 3 instead of just 2 sublayers.



The decoder [Image taken from [Attention Is All You Need](#), Vaswani et al. (2017)]

The first sublayer receives the decoder's output from the previous time step, enhanced with the positional encoding. Once again, we implement the multi-head self-attention mechanism. However, this time we only attend to the preceding words and not to all words in the sequence simultaneously. This ensures that the prediction at the current time step can only depend on the previous words and is not based on, otherwise, illegal connections into the future. We achieve this effect by implementing a mask that suppresses the relevant matrix values accordingly. This specifies our decoder as unidirectional.

The second sublayer implements a multi-head self-attention layer similar to the encoder. This layer receives the queries from the previous sublayer and the keys and values from the encoder's output.

The final layer is a fully connected feed-forward network which is basically the same as the one implemented in the encoder.

Moreover, we implement residual connections and apply a normalization layer after each sublayer.

The overall model

Now, that we covered both the encoder and decoder we can assemble the different pieces into an overall model that runs as follows:

First of all, we create an embedding vector for each word in the sequence. Next, we augment this embedding with the positional encoding. This input is fed into the encoder and we proceed with the two sublayers as described earlier.

Now, the decoder receives its previous output also enhanced by the positional encoding. This input is routed into the three sublayers. We apply a mask in the first sublayer to suppress illegal connections. In the second sublayer, we receive the encoder's output alongside the output from the decoder's first sublayer and pass that through a fully-connected neural network to generate the decoder's final output.

In the end, we route the decoder's output through another fully-connected neural network and apply a softmax function to generate a prediction for the next word in the sequence.

Conclusion

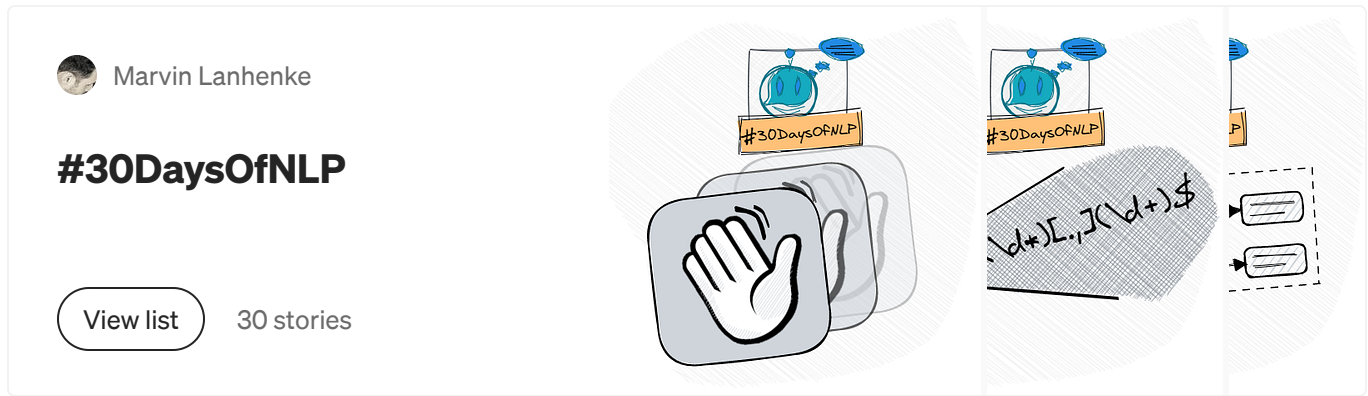
In this article, we shifted our focus from the attention mechanism to the Transformer architecture itself. We covered the fundamental building blocks of the encoder and decoder. We learned how each block is composed of multiple sublayers and how the information flows through the complete architecture.

Now, it's time to apply our knowledge.

It's time to implement a Transformer model ourselves.

And this is what we will do in the next episode.

So prepare your IDE, don't go anywhere, make sure to follow, and never miss a single day of the ongoing series **#30DaysOfNLP**.



Enjoyed the article? Become a Medium member and continue learning with no limits. I'll receive a portion of your membership fee if you use the following link, at no extra cost to you.

Join Medium with my referral link — Marvin Lanhenke

As a Medium member, a portion of your membership fee goes to writers you read, and you get full access to every story...

medium.com

References / Further Material:

- <https://arxiv.org/abs/1706.03762>

Mlearning.ai Submission Suggestions

How to become a writer on Mlearning.ai

medium.com

Naturallanguageprocessing

NLP

Transformers

Deep Learning

MI So Good

**Written by Marvin Lanhenke**

956 followers · 48 following

Follow

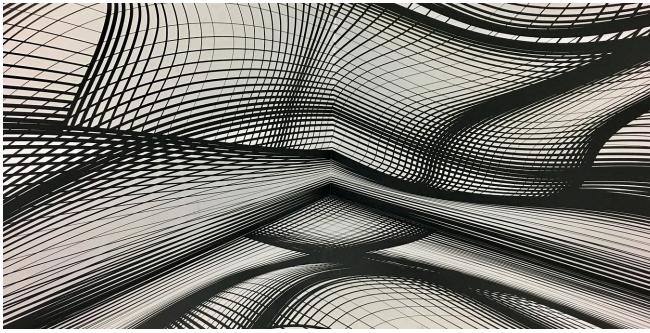
Business Analyst. Solutions Architect. Self-Taught. Hands-On. Writing about Software Architecture & Engineering. Say Hi @ [linkedin.com/in/marvinlanhenke/](https://www.linkedin.com/in/marvinlanhenke/)

No responses yet

Alejandro Jofre

What are your thoughts?

More from Marvin Lanhenke



In TDS Archive by Marvin Lanhenke

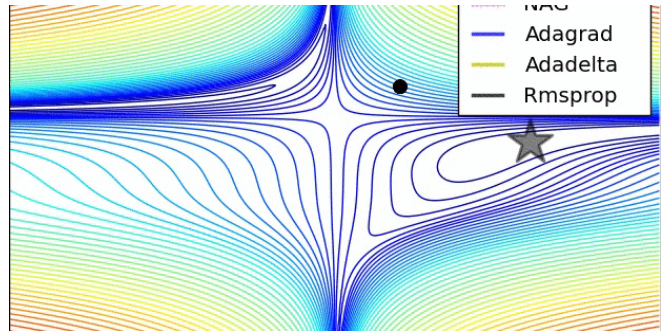
Implementing Support Vector Machine From Scratch

Understanding the maximal margin classifier with gradient descent and hinge loss by...

Jan 24, 2022

👏 200

💬 2



Musstafa

Optimizers in Deep Learning

What is an optimizer?

Mar 27, 2021

👏 446

💬 1



Maximilian Vogel

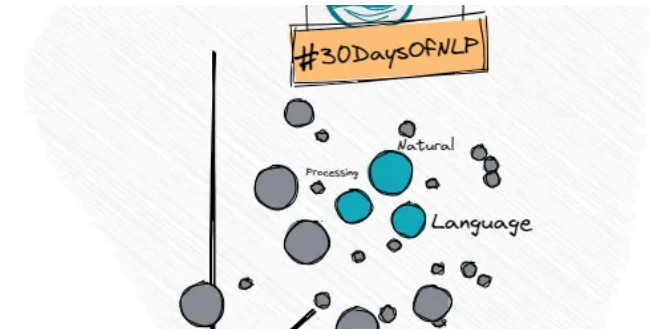
I Scanned 1000+ Prompts so You Don't Have to: 10 Need-to-Know...

Free Prompt Engineering Course: The Art of the Prompt — Updated Dec-10 2023, added...

Aug 11, 2023

👏 2.4K

💬 29



Marvin Lanhenke

NLP-Day 27: How To Visualize Word Embeddings With Tensorboard

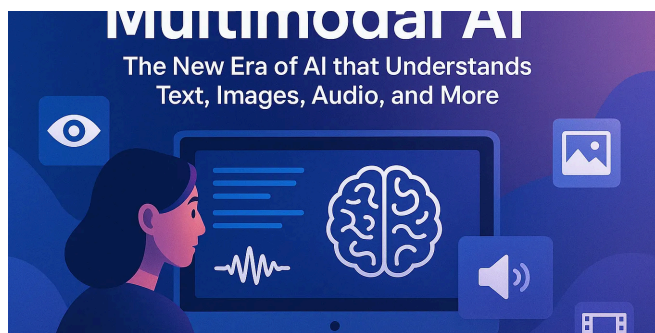
A picture is worth more than a 1000 word embedding

May 3, 2022

👏 69

[See all from Marvin Lanhenke](#)

Recommended from Medium



In Towards AI by Rohan Mistry

Multimodal AI: The New Era of AI that Understands Text, Images,...

What if your AI assistant could watch a video, read an article about it, listen to a podcast...



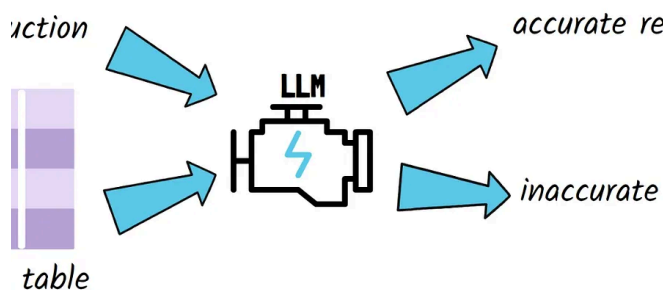
Oct 3



5



1



Jamalla Zawia

How Do LLMs Really Understand Your Prompts?

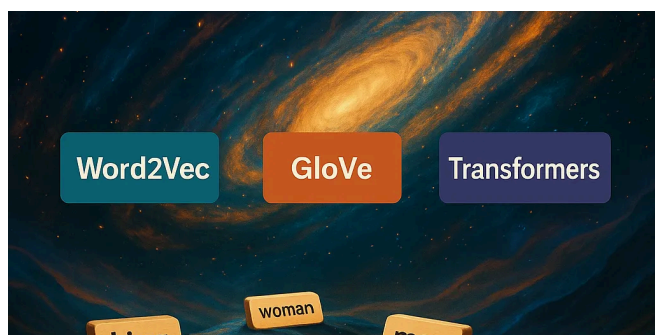
I am sure you have read many articles that suggest using phrases like “act as” or...



Aug 5



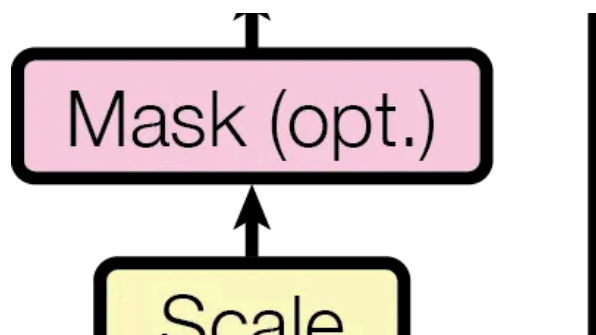
4



Seema Nair

Word2Vec, GloVe, and Transformers: A Journey Through...

In the world of Natural Language Processing (NLP), how we represent words fundamental...



Kari Vierimaa

Transformers From Scratch: Part 2 —Scaled Dot-Product Attention

Builds the core Scaled Dot-Product Attention mechanism, including masking and scaling.

★ Jul 8

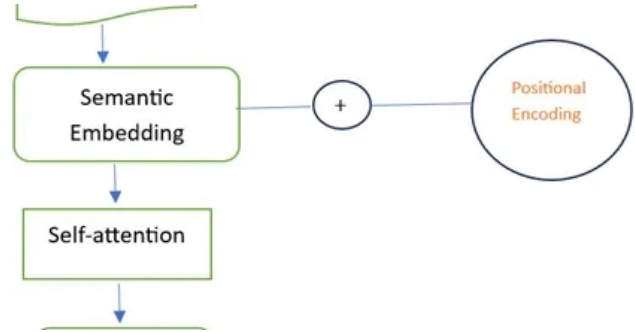


...

Apr 12 1



...



In Foundation Models Deep Dive by M

Positional Embeddings in LLMs

Giving LLMs a sense of order.

★ Jun 7



...

Jul 18



...



Pankaj Agrawal

Semantic Embeddings, Contextual Embeddings and Self-Attention

Embeddings convert raw textual data into meaningful numerical vectors, fundamentall...

[See more recommendations](#)